

List

The list is a most versatile datatype available in Python which can be written as a list of comma-separated values (items) between square brackets. Important thing about a list is that items in a list need not be of the same type.

Creating a list is as simple as putting different comma-separated values between square brackets. For example

```
list1 = ['physics', 'chemistry', 1997, 2000];  
list2 = [1, 2, 3, 4, 5 ];  
list3 = ["a", "b", "c", "d"]
```

Similar to string indices, list indices start at 0, and lists can be sliced, concatenated and so on.

Some objects contain references to other objects, these objects are called containers. Some examples of containers are a tuple, list, and dictionary. The value of an **immutable container** that contains a reference to a **mutable object** can be changed if that mutable object is changed. Mutable vs Immutable Objects in Python. ... Simple put, a mutable object can be changed after it is created, and an immutable object can't. Objects of built-in types like (int, float, bool, str, tuple, unicode) are immutable. Objects of built-in types like (list, set, dict) are mutable.

Arrays and lists are both used in Python to store data, but they don't serve exactly the same purposes. They both can be used to store any data type (real numbers, strings, etc), and they both can be indexed and iterated through, but the similarities between the two don't go much further. The main difference between a list and an array is the functions that you can perform to them. For example, you can divide an array by 3, and each number in the array will be divided by 3 and the result will be printed if you request it. If you try to divide a list by 3, Python will tell you that it can't be done, and an error will be thrown. It does take an extra step to use arrays because they have to be declared while lists don't because they are part of Python's syntax, so lists are generally used more often between the two, which works fine most of the time. However, if you're going to perform arithmetic functions to your lists, you should really be using arrays instead. Additionally, arrays will store your data more compactly and efficiently, so if you're storing a large amount of data, you may consider using arrays as well.

Problem Statement:

1. Do the following in Python shell.

a) Save a list. Print all its values without using a loop.

```
>>> list=[1,2,3]  
>>> print(list)
```

b) Save a list and delete one item from it. Now print it.

```
>>> list=[1,2,3,4,5,6]  
>>> del list[3]  
>>> print(list)
```

c) Concatenate two lists.

```
>>> l1=[1,2,3,4]
>>> l2=[10,20,30]
>>> l1.extend(l2)
>>> print(l1)
```

- d) Store items in list using list.

```
>>> list=[x for x in range(5)]
>>> print(list)
```

- e) Save a list. Print i^{th} to j^{th} item without using loop.

```
>>> l1=[1,2,3,4]
>>> print(l1[1:3])
```

- f) Convert a tuple in a list.

```
>>>l2=[10,20,30]
>>> tuple(l2)
(10, 20, 30)
```

- g) Find maximum and minimum item in a list. Find the length of a list.

```
>>>l2=[10,20,30]
>>> max(l2)
>>> min(l2)
>>> len(l2)
```

- h) Create two lists. Append an item. Append a tuple of length two. Concatenate two lists. Find index of an item. Insert 'six' at index 5. Delete last item. Delete item with index 4. Remove the item 'six'. Reverse the list and sort it. Print the results at each step.

```
>>> l1=[1,2,3]
>>> l2=[11,12,13]
>>> l1.append(4)
>>> print(l1)
>>> l1.append(('5a','5b'))
>>> print(l1)
>>> l1.extend(l2)
>>> print(l1)
>>> print(l2)
>>> l1.index(11)
>>> l1.insert(5,'six')
>>> print(l1)
>>> l1.pop()
>>> print(l1)
>>> l1.pop(4)
>>> print(l1)
>>> l1.remove('six')
>>> print(l1)
>>> l1.reverse()
>>> print(l1)
>>> l1.sort()
>>> print(l1)
```

- i) Check whether an item belongs to the list.

```
>>> my_list = ['p','r','o','b','l','e','m']
```

```
>>> print('p' in my_list)
>>> print('a' in my_list)
>>> print('c' not in my_list)
```

j) Create list of cube of first ten natural numbers in a single line.

```
>>> cubes=[x**3 for x in range(10)]
>>> print(cubes)
```

2. Write a program in Python to create histogram of values from a list of values.

```
values = [] # a list of values
# input 10 values from user
print "Enter 10 integers:"
for i in range( 10 ):
    newValue = int(input( ( i + 1 ) ))
    values += [ newValue ]
# create histogram
print "\nCreating a histogram from values:"
print "%s %10s %10s" % ( "Element", "Value", "Histogram" )
for i in range( len( values ) ):
    print "%7d %10d %s" % ( i, values[ i ], "*" * values[ i ] )
```

Output:

```
Enter 10 integers:
Enter integer 1: 19
Enter integer 2: 3
Enter integer 3: 15
Enter integer 4: 7
Enter integer 5: 11
Enter integer 6: 9
Enter integer 7: 13
Enter integer 8: 5
Enter integer 9: 17
Enter integer 10: 1
```

Creating a histogram from values:

Element	Value	Histogram
0	19	*****
1	3	***
2	15	*****
3	7	*****
4	11	*****
5	9	*****
6	13	*****
7	5	*****
8	17	*****
9	1	*

1. Create a table using list of lists by writing a program in Python.

```
table1 = [ [ 1, 2, 3 ], [ 4, 5, 6 ] ]
print "Values in table1 by row are"
for row in table1:
    for item in row:
        print item,
```

Output:

Values in table1 by row are

1 2 3

4 5 6

2. Write a Python program that prompts the user to enter a string and returns in alphabetical order, a letter and its frequency of occurrence in the string (ignore case).
3. Write a Python program to add two matrices.
4. Write a Python program which creates a list of numbers. Split the list into two lists one containing all even numbers and another all odd numbers.
5. Create a list of strings. Write a Python program which creates another list from the first taking the first character from each word.
6. Write a Python program that passes a list to a function that squares each element in the list. Print the list values at different stages to show the changes made to one list is automatically reflected in the other list.
7. Implement stack and queue.